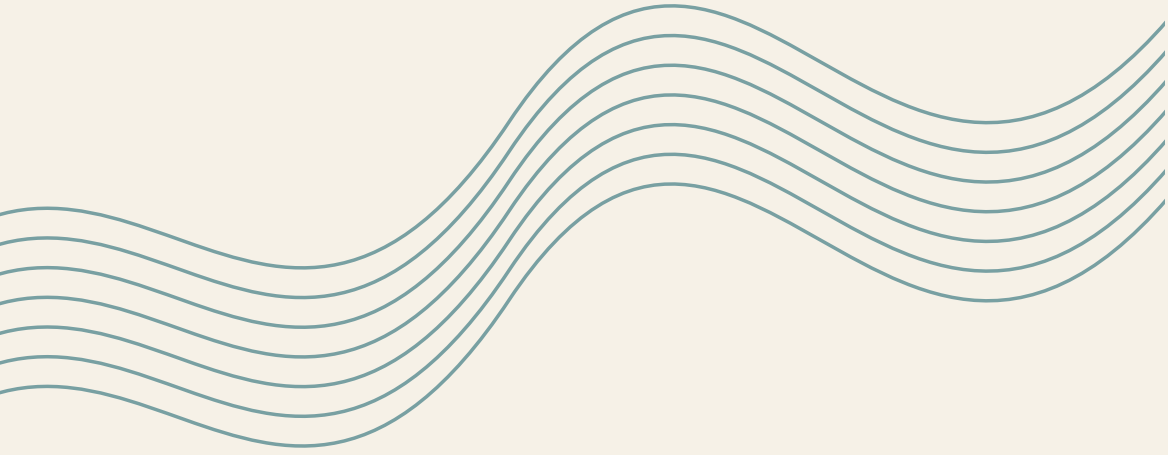
복구된 소스에서 실행의 질서를 읽다

Claude Code v2.1.88 스냅샷의 turn, stream, effect, transcript, topology를 한 시스템으로 이해하는 기술 독본

공개 저장소의 제3자 복구 스냅샷을 분석한다. 공식 Anthropic 설계나 현재 제품 전체를 설명한다고 주장하지 않는다.

고정 revision
2ca5ddab…
tree 1b6b4ab…



SUCCESSOR AUTHORING CANDIDATE
SYSTEMS MONOGRAPH × EVIDENCE NOTEBOOK
SEARCHABLE A4 · 2026-09-21

Reader orientation

이 보고서는 제품 설명서가 아니라 고정된 소스의 작동 모델이다

독자의 목표는 파일 이름을 외우는 것이 아니라, 입력이 상태를 바꾸고 효과를 만들며 기록과 복구로 닫히는 경계를 이해하는 것이다.

● observed_project_fact

분석 대상은 commit `2ca5ddabfed5…`, tree `1b6b4ab07d0…`로 고정된 공개 저장소의 복구 스냅샷이다. manifest는 1,952개 tracked file, 1,884개 TypeScript/TSX 구현 파일, 512,664 physical TypeScript line을 기록한다. build와 runtime은 실행하지 않았다.

저장소가 package 2.1.88에서 복구했다고 서술하는 사실은 *repository-authored* 주장이다. 누락 모듈과 feature gate가 존재할 수 있으므로 현재 공식 제품과의 parity를 추정하지 않는다.

PROJECT-ANALYSIS.md · source-state.json · artifact-manifest.json

? unresolved **이 보고서가 답하지 않는 것**

- 공식 upstream 의도와 현행 제품 전체 기능
- 실제 runtime hot path와 장애율
- 공식 module owner 또는 bounded context
- 구현·리팩터링·출판의 승인 여부

같은 코드도 “무엇을 관찰했는가”와 “무엇을 해석했는가”를 나누면 훨씬 정확하게 읽힌다.

01 상태와 실행의 한 줄 구조 04–08

02 열두 알고리즘의 경계 불변식 09–20

03 물리 topology와 의미 모듈 후보 22–26

04 관리 · 증거 · 점진적 모듈화 27–31

05 미해결 질문과 근거 색인 32–37

Provenance and authority

네 종류의 문장을 같은 목소리로 쓰지 않는다

원천 분석의 *source-observed*는 이 책에서 *observed_project_fact*로 번역한다. 나머지 문장은 배경, 추론, 미해결 상태를 명시한다.

● observed_project_fact

고정 snapshot에서 직접 확인

source path, line locator, 정적 import 수, algorithm transition처럼 현재 bundle의 근거가 직접 지지한다. 단, 정적 관찰은 runtime reachability를 증명하지 않는다.

▤ external_background

이해에 필요한 일반 개념

generator의 pause/resume, client tool loop처럼 공식 기술 문서가 설명하는 배경이다. 배경 개념은 프로젝트 구현의 증거가 아니며 적용 범위를 따로 적는다.

◆ project_inference

사실과 배경을 잇는 명시적 해석

예를 들어 여덟 의미 모듈은 현행 package가 아니라 책임을 따라 읽기 위한 분석 가설이다. 반증 조건과 extraction risk를 함께 둔다.

? unresolved

관찰 범위 밖의 질문

누락된 feature-gated source, 공식 ownership, production behavior, runtime trace는 보이지 않는다. 일반 튜토리얼로 빈칸을 채우지 않는다.

> **Authority 경계** — candidate 작성, worker contribution, PDF 생성은 activation·acceptance·publication·integration·settlement가 아니다.

Prerequisite bridge

상태 기계, 효과 경계, 기록 그래프

프로젝트의 긴 함수들을 이해하려면 코드 문법보다 “무엇이 상태이고, 누가 전이를 허용하며, 무엇이 다시 읽힐 기록이 되는가”를 먼저 구분해야 한다.

▤ external_background

중단 가능한 상태 기계

generator는 return으로 끝내기 전에 yield로 frame을 보존할 수 있다. 여기서는 문법 자체보다 suspended·running·completed 사이의 제어 전달이 중요하다.

▤ external_background

요청과 효과의 분리

모델의 tool call은 외부 세계를 바꾼 결과가 아니다. client가 schema, permission, executor를 통과시킨 뒤 tool_result를 다음 요청에 돌려준다.

◆ project_inference

로그가 아니라 복구 그래프

JSONL append와 parentUuid 복구를 함께 보면 transcript는 단순 출력 보관이 아니라 다음 turn과 resume를 재구성하는 graph journal로 읽을 수 있다.



이 보고서는 모든 결정적 알고리즘을 같은 여섯 질문으로 읽는다: 무엇이 시작하고, 어떤 상태를 보유하며, 어떻게 전이하고, 무엇을 지켜야 하며, 어디서 실패하고, 어떻게 닫는가.

다음 단계 세 개념을 실제 실행 흐름에 겹치면 input, model stream, tool effect, transcript가 하나의 순환 구조로 연결된다.

One-sentence architecture

이 시스템의 중심은 채팅 화면이 아니라 결과를 다시 입력으로 만드는 상태 기계다

입력은 분류되고, turn은 provider stream을 소비하며, 허용된 tool effect를 실행하고, 결과와 transcript를 다음 turn 또는 resume가 읽을 수 있는 형태로 닫는다.

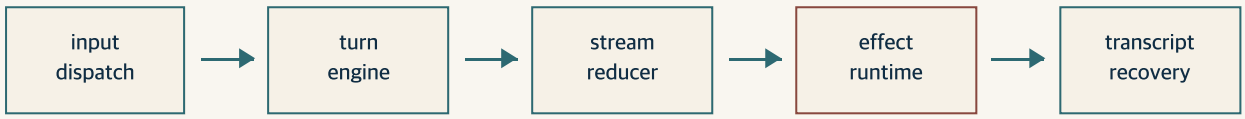


그림 1. 제안된 tool use는 effect runtime을 통과해야 하며, tool_result와 transcript가 다시 turn engine의 문맥이 된다.

제어의 왕복

`src/query.ts`의 generator loop는 message, turn count, compaction과 pending tool state를 보유한다. `src/services/api/claude.ts`는 provider event를 내부 content block과 usage·stop reason으로 줄인다.

tool proposal은 `toolExecution.ts`와 `toolOrchestration.ts`에서 validation, hook, permission, scheduling을 통과한다. 결과는 protocol-compatible `tool_result`로 닫혀 다음 model turn에 재주입된다.

흐름을 지키는 불변식

핵심은 함수의 개수가 아니라 짝과 순서다. tool_use에는 tool_result가 필요하고, 실패한 fallback의 결과는 폐기되어야 하며, abort도 미완료 result를 닫아야 한다.

session journal은 append 순서를 보존하고 parent topology를 다시 만들 수 있어야 한다. compaction은 정보를 잃는 대신 무엇을 보존했는지를 명시해야 한다.

> **전환** 다음 두 면은 같은 사용자 요청을 따라가며 각 알고리즘이 언제 소유권을 넘기는지 추적한다.

End-to-end sequence

하나의 요청은 일곱 번 의미가 바뀐다

문자열은 입력 intent가 되고, internal message가 되고, provider event와 tool proposal을 거쳐 durable record와 다음 turn의 문맥이 된다.

단계	소유 상태	넘기는 값	깨지기 쉬운 경계
01 입력 분류	mode, content block, origin	prompt 또는 control action	remote command safety
02 대화 투영	messages, session, abort	internal message + context	checkpoint ordering
03 turn loop	turn/recovery budget	model request	terminal reason, abort
04 stream reduce	partial blocks, usage	assistant slices / tool use	late finalization
05 effect 해석	schema, permission, hooks	tool_result	denial vs execution
06 transcript append	UUID, parent, write queue	durable JSONL entries	ordered flush
07 resume/continue	graph, leaf, compact boundary	reconstructed context	cycle/orphan repair

◆ project_inference

이 흐름을 "요청 → 응답"으로만 그리면 가장 중요한 authority와 durability 경계가 사라진다.

따라서 이 책은 UI, model, tool을 독립 기능 카드로 나열하지 않고, 각 단계가 다음 단계에 넘기는 값과 실패했을 때 닫아야 하는 protocol을 함께 표시한다.

Concrete example

“설정 파일의 값을 바꿔 줘” 라는 요청

예시는 실제 prompt 내용을 재현하지 않고, source에서 관찰된 경계만 사용해 한 요청의 state와 authority 이동을 설명한다.

STEP 01

입력을 분류한다

`processUserInputBase`는 mode와 origin을 확인하고 media와 attachment를 정규화한다. remote bridge가 허용되지 않은 local command를 보내면 model query 전에 차단한다.

STEP 02

대화 owner가 turn 입력을 만든다

`QueryEngine`은 external request를 internal message로 바꾸고 session identity와 abort controller가 들어 있는 context를 결합한다.

STEP 03

generator turn을 시작한다

`query()`는 message와 ToolUseContext를 받고 model stream을 연다. turnCount와 recovery budget은 이 실행이 무한히 반복되지 않게 한다.

STEP 04

partial stream을 내부 block으로 줄인다

content block의 index와 type을 검증하며 delta를 누적한다. file edit tool_use가 완성되면 아직 외부 파일은 바뀌지 않았다.

STEP 05

효과를 허가할 수 있는지 판단한다

tool lookup, schema validation, pre-hook, permission을 통과한 요청만 executor로 간다. denial도 protocol-compatible tool_result로 돌아간다.

계속 다음 면은 file edit의 stale check와 commit point, tool_result closure, transcript append, resume 가능 상태까지 이어진다.

TRACE-E2E · CONTINUED · STEPS 06–10

효과의 완료와 다음 turn의 시작

현재 상태: tool identity와 input은 검증되었고 permission은 allow다. 아직 성공을 주장할 수 없으며 filesystem commit과 결과 closure가 남아 있다.

STEP 06

읽었던 파일이 아직 같은지 확인한다

FileEditTool은 expected string과 match cardinality를 검사하고 동기적으로 다시 읽는다. stale이면 write 전에 실패한다.

STEP 07

temporary file을 flush하고 rename을 시도한다

mode와 symlink behavior를 가능한 범위에서 보존한다. atomic replace가 실패하면 flushed direct write로 fallback하지만 cleanup은 best effort다.

STEP 08

결과를 tool_result로 닫는다

성공·실패·거부·abort는 서로 다른 의미를 유지한 채 result가 된다. turn loop는 누락된 result가 있으면 합성해 pairing invariant를 회복한다.

STEP 09

journal에 순서대로 append한다

message UUID와 parent cursor를 연결하고 per-file queue가 JSONL line 순서를 보존한다. 이 기록은 다음 model request와 resume의 근거다.

STEP 10

다음 turn 또는 복구 경로를 선택한다

정상 경로라면 result를 다시 model context로 넣는다. 재개라면 parent graph, compact segment, orphan sibling을 repair한 뒤 bounded chain을 구성한다.

> **예제의 한계** — 이 sequence는 catalog의 관찰된 경계를 연결한 설명용 inference다. 실제 runtime trace나 production latency를 관찰한 기록은 아니다.

Reader question

Generator 기반 turn 상태 기계

한 turn은 언제 계속되고 언제 끝나는가?

● observed_project_fact

query()는 model stream과 tool 결과를 소비하면서 bounded state를 교체하고 terminal reason을 산출한다.

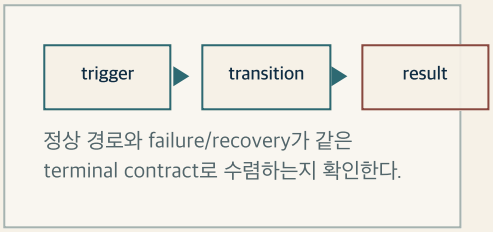
긴 대화를 단일 함수 호출로 보면 partial observation, tool execution, abort 이후 closure를 설명할 수 없다. 중단 가능한 실행 frame과 명시적 terminal state가 필요하다.

`messages` `toolUseContext` `turnCount` `compaction` `output recovery` `stop hooks` `pending tools`

Trigger	query()가 initial messages와 ToolUseContext를 받음
Transition	model event를 stream하고 tool_use를 기록한다. tool을 실행하거나 drain하고 abort/error에서 누락 result를 합성한 뒤 terminal reason을 반환하거나 새 State로 계속한다.
Invariant	• 모든 tool_use는 tool_result로 닫힌다 • turn과 recovery budget은 bounded다 • 실패한 fallback attempt의 결과는 폐기한다
Failure / recovery	• model/runtime error는 terminal result가 된다 • abort는 tool result를 drain 또는 합성한다 • prompt/output limit는 bounded recovery를 거친다 • maxTurns는 max_turns로 끝난다

◆ why it matters

이 경계를 알면 UI에 text가 보이는 순간과 turn이 protocol상 완료된 순간을 구분할 수 있다.



Exact project evidence

src/query.ts:219 · src/query.ts:241 · src/query.ts:554 · src/query.ts:834 · src/query.ts:1012 · src/query.ts:1062 · src/query.ts:1381 · src/query.ts:1506 · src/query.ts:1705

> 정적 source 관찰이며 runtime reachability와 current-product parity는 검증하지 않았다.

Reader question

Tool proposal을 허가된 effect로 해석하기

모델의 요청은 언제 외부 세계를 바꿀 수 있는가?

● observed_project_fact

runToolUse는 schema, hook, permission과 executor를 순서대로 통과시켜 proposal과 effect 사이에 authority boundary를 만든다.

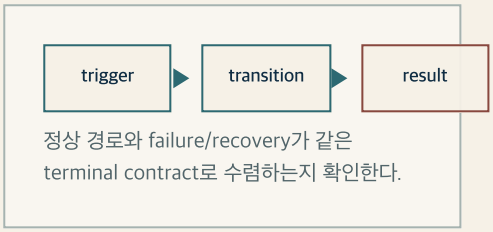
tool 이름과 JSON input이 있다는 사실은 실행 권한이 있다는 뜻이 아니다. 거부도 다음 turn이 이해할 수 있는 result여야 한다.

`tool schema` `processed input` `pre hooks` `permission` `progress` `post hooks` `abort signal`

Trigger	streamed ToolUseBlock이 runToolUse에 전달됨
Transition	tool을 찾고 input을 검증한다. hook와 permission을 적용해 거부 result를 만들거나 실행하고 progress, success, error를 정규화한다.
Invariant	• unknown/invalid tool은 실행하지 않는다 • non-allow decision은 effect boundary를 넘지 않는다 • 모든 경로가 protocol-compatible result를 시도한다
Failure / recovery	• hook와 permission denial은 reason을 반환한다 • AbortError와 tool failure를 구분한다 • post-failure hook는 retry guidance만 더하고 직접 retry하지 않는다

◆ why it matters

모델 능력과 client 권한을 같은 것으로 오해하지 않게 하며, audit에서 누가 실제 효과를 허용했는지 찾을 수 있다.



정상 경로와 failure/recovery가 같은 terminal contract로 수렴하는지 확인한다.

Exact project evidence

src/services/tools/toolExecution.ts:337 · src/services/tools/toolExecution.ts:599 · src/services/tools/toolExecution.ts:916 · src/services/tools/toolExecution.ts:995 · src/services/tools/toolExecution.ts:1571 · src/services/tools/toolHooks.ts:39 · src/services/tools/toolHooks.ts:332 · src/services/tools/toolHooks.ts:435

> 정적 source 관찰이며 runtime reachability와 current-product parity는 검증하지 않았다.

Reader question

Provider stream reducer와 late finalization

화면에 보인 partial text는 언제 canonical message가 되는가?

● observed_project_fact

event reducer는 indexed content block을 누적하고, 이미 yield한 object에 late usage와 stop reason을 반영해 transcript와 terminal state를 완성한다.

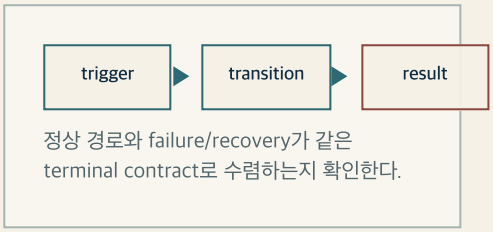
streaming은 문자열을 이어 붙이는 기능이 아니다. block identity, delta type, usage와 종료 정보가 서로 다른 시점에 도착한다.

`partial message` `indexed blocks` `usage` `stop reason` `yielded references` `watchdog/fallback`

Trigger	queryModelWithStreaming이 API event stream을 엶
Transition	block_start가 state를 만들고 delta가 typed fragment를 추가한다. block_stop은 assistant slice를 yield하고 message_delta는 이미 yield한 object의 final usage와 stop reason을 갱신한다.
Invariant	• delta type과 target block type이 일치한다 • content-block index가 존재한다 • late usage/stop data가 queued transcript reference에 도달한다
Failure / recovery	• event 순서가 잘못되면 명시적 error • idle stream abort는 non-streaming fallback • user abort는 model failure로 보고하지 않는다

◆ why it matters

partial UI projection과 durable canonical message를 분리해야 resume와 usage accounting이 일치한다.



Exact project evidence
src/services/api/claude.ts:752 · src/services/api/claude.ts:1017 · src/services/api/claude.ts:1995 · src/services/api/claude.ts:2053 · src/services/api/claude.ts:2213 · src/services/api/claude.ts:2229 · src/services/api/claude.ts:2309

> 정적 source 관찰이며 runtime reachability와 current-product parity는 검증하지 않았다.

Reader question

Queued append-only JSONL journal

대화 기록은 어떤 순서로 durable state가 되는가?

● observed_project_fact

per-file write queue는 UUID와 parent cursor를 연결한 entry를 순서대로 batch append하여 종료와 resume가 공유할 journal을 만든다.

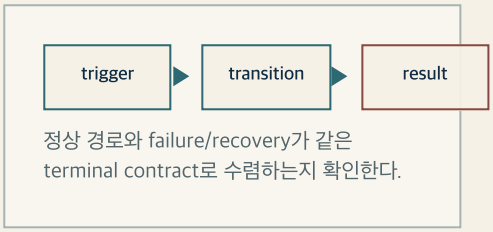
화면 로그는 사라져도 되지만 resume 입력은 순서와 parent relation을 잃으면 안 된다. durability는 단순 file write보다 강한 계약이다.

per-file queue | flush timer | session file | UUID set | parent cursor | remote state

Trigger	message, queue operation, metadata 또는 recovery record가 persistence에 admit됨
Transition	message chain에 parent를 부여하고 entry kind를 route한다. serialized line을 queue에 넣고 flush가 파일별 순서대로 append한다.
Invariant	• main-chain UUID dedup과 sidechain local persistence가 다르다 • 파일별 entry 순서를 보존한다 • progress message는 parent chain에 참여하지 않는다
Failure / recovery	• shutdown에서 flush를 await할 수 있다 • 누락 directory는 append 시 만든다 • remote persistence와 local sidechain dedup을 분리한다

◆ why it matters

다음 turn과 resume가 같은 history를 읽으려면 write order와 graph identity를 하나의 contract로 관리해야 한다.



Exact project evidence
src/utils/sessionStorage.ts:532 · src/utils/sessionStorage.ts:606 · src/utils/sessionStorage.ts:634 · src/utils/sessionStorage.ts:841 · src/utils/sessionStorage.ts:993 · src/utils/sessionStorage.ts:1128 · src/utils/sessionStorage.ts:1408

> 정적 source 관찰이며 runtime reachability와 current-product parity는 검증하지 않았다.

Reader question

Transcript graph 재구성과 repair

중단된 session을 어떤 history로 다시 시작하는가?

● observed_project_fact

resume loader는 JSONL을 UUID graph로 만들고 compact · snip · legacy gap · parallel sibling을 repair해 bounded leaf-to-root chain을 구성한다.

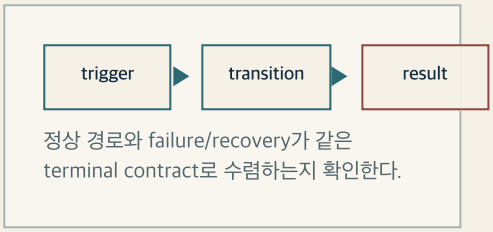
append 순서만으로는 branch와 compaction을 복원할 수 없다. parent link와 보존 구간, 삭제 구간을 graph transform으로 다뤄야 한다.

`UUID map` `parentUuid` `compact segments` `snip removals` `candidate leaves` `parallel siblings`

Trigger	persisted JSONL transcript를 resume, branch, share 또는 agent recovery로 읽음
Transition	record를 parse하고 legacy gap을 bridge한다. preserved segment를 relink하고 snipped range를 제거·연결한 뒤 cycle detection으로 leaf-to-root를 걷고 orphan sibling/result를 회수한다.
Invariant	• cycle에서는 bounded partial chain을 반환한다 • 삭제된 middle range는 dangling parent를 남기지 않는다 • parallel sibling은 API pairing 가능성을 유지한다
Failure / recovery	• malformed preserved metadata는 fuller history로 fallback • resume consistency가 expected/actual delta를 낸다 • legacy record는 read-side repair한다

◆ why it matters

resume는 파일을 다시 여는 행위가 아니라, 다음 model request가 사용할 일관된 대화 graph를 재구성하는 일이다.



Exact project evidence
src/utils/sessionStorage.ts:1839 · src/utils/sessionStorage.ts:1982 · src/utils/sessionStorage.ts:2069 · src/utils/sessionStorage.ts:2097 · src/utils/sessionStorage.ts:2224 · src/utils/sessionStorage.ts:3472 · src/utils/sessionStorage.ts:3704

> 정적 source 관찰이며 runtime reachability와 current-product parity는 검증하지 않았다.

Reader question

Provider message compiler와 tool-pair repair

내부 transcript는 어떻게 provider가 받을 수 있는 message가 되는가?

● observed_project_fact

compiler pass는 내부 variant를 정규화 · 병합하고 tool pair를 수리한 뒤 provider-incompatible metadata를 제거한다.

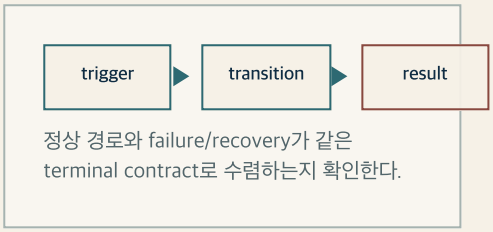
내부 기록은 UI와 recovery에 필요한 정보를 포함하지만 provider API는 더 좁은 schema와 pairing 규칙을 요구한다.

`normalized sequence` `assistant merge` `tool-use IDs` `tool-result IDs` `media/cache policy`

Trigger	internal transcript message를 provider API MessageParam으로 변환해야 함
Transition	internal message를 normalize/merge하고 unsupported field를 제거한다. orphan tool use에 synthetic error result를 넣고 orphan/duplicate result를 제거한 뒤 cache marker를 추가한다.
Invariant	• API 제출 전 tool ID가 pair된다 • provider-incompatible metadata를 제거한다 • cache annotation은 reusable source block을 in-place mutate하지 않는다
Failure / recovery	• missing result는 synthetic error result • orphan result는 제거 • repair 후 empty message는 bounded fallback

◆ why it matters

durable transcript와 provider message를 같은 자료구조로 가정하지 않아야 repair와 information loss를 명시할 수 있다.



Exact project evidence
src/services/api/claude.ts:1266 · src/services/api/claude.ts:1298 · src/utils/messages.ts:1989 · src/utils/messages.ts:5123 · src/utils/messages.ts:5305 · src/services/api/claude.ts:3063

> 정적 source 관찰이며 runtime reachability와 current-product parity는 검증하지 않았다.

Reader question

입력 의존 tool scheduling과 deterministic fold

여러 tool을 언제 병렬로 실행해도 되는가?

● observed_project_fact

scheduler는 연속 safe call을 bounded concurrent batch로 묶고 unsafe call을 singleton으로 직렬화한 뒤 context modifier를 원래 순서로 fold한다.

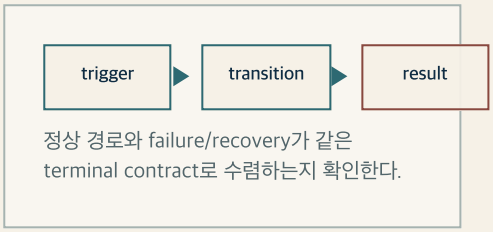
동시 실행은 속도만의 문제가 아니다. 두 tool이 공유 state를 바꾸면 완료 순서가 context 의미를 바꿀 수 있다.

`ordered calls` `safety classification` `batches` `ToolUseContext` `context modifiers` `in-progress IDs`

Trigger	한 assistant turn이 하나 이상의 tool call을 생성함
Transition	safe consecutive call은 concurrent batch, unsafe call은 serial singleton으로 partition한다. cap 아래 실행 후 original block order로 modifier를 적용한다.
Invariant	• classification error는 serial로 fail closed • unsafe tool은 concurrent batch를 공유하지 않는다 • context mutation은 batch 뒤 결정적으로 적용한다
Failure / recovery	• schema parse failure는 non-concurrent • tool failure는 기본적으로 scheduling abort가 아닌 result • 완료 시 in-progress ID를 제거한다

◆ why it matters

실행 완료 순서와 state 반영 순서를 분리하면 성능을 얻으면서도 다음 turn의 context를 재현할 수 있다.



Exact project evidence
src/services/tools/toolOrchestration.ts:8 · src/services/tools/toolOrchestration.ts:19 · src/services/tools/toolOrchestration.ts:91 · src/services/tools/toolOrchestration.ts:118 · src/services/tools/toolOrchestration.ts:152 · src/services/tools/StreamingToolExecutor.ts

> 정적 source 관찰이며 runtime reachability와 current-product parity는 검증하지 않았다.

Reader question

Bounded retry를 가진 lossy context migration

context가 너무 커질 때 무엇을 잃고 무엇을 보존하는가?

● observed_project_fact

compaction은 older context를 summary와 restoration attachment로 이동하고 retry와 연속 실패를 bounded budget으로 제한한다.

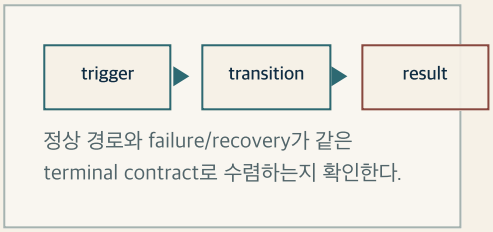
요약은 투명한 압축이 아니라 정보 손실을 수반하는 migration이다. 보존한 것과 버린 것을 boundary metadata로 드러내야 한다.

`token estimate` `threshold/reserve` `failure counter` `summarize/keep sets` `summary/attachments` `boundary metadata`

Trigger	estimated token이 auto/manual threshold를 넘거나 recovery가 compaction을 요청함
Transition	eligibility를 판단하고 older context를 summarize한다. prompt-too-long이면 oldest API-round group을 줄여 retry하고 boundary와 restoration attachment를 emit한다.
Invariant	• compact/session-memory query는 recursive autocompact하지 않는다 • retry와 consecutive failure는 bounded • preserved context를 명시적으로 표현한다
Failure / recovery	• auto-compact 3회 연속 실패면 circuit open • prompt-too-long retry 최대 3회 • manual과 automatic failure presentation이 다르다

◆ why it matters

context limit을 넘는 순간을 단순 error가 아니라 provenance가 있는 migration으로 관리하면 resume와 debugging이 가능해진다.



Exact project evidence
src/services/compact/autoCompact.ts:70 · src/services/compact/autoCompact.ts:160 · src/services/compact/autoCompact.ts:241 · src/services/compact/compact.ts:227 · src/services/compact/compact.ts:243 · src/services/compact/compact.ts:387 · src/services/compact/compact.ts:713

> 정적 source 관찰이며 runtime reachability와 current-product parity는 검증하지 않았다.

Reader question

Optimistic file edit와 atomic-replace 시도

읽은 뒤 바뀐 파일에 오래된 편집을 쓰지 않으려면?

● observed_project_fact

FileEditTool은 expected content와 mtime을 재확인하고 await 없는 commit window에서 temp flush · rename을 시도한다.

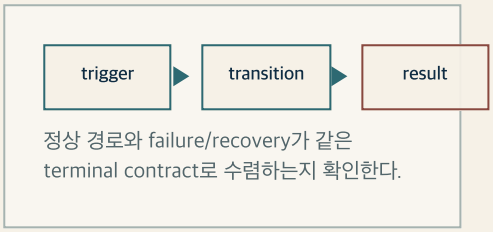
read와 write 사이에 다른 actor가 파일을 바꿀 수 있다. edit tool은 자신이 본 state가 여전히 유효한지 증명해야 한다.

`last-read content/time` `current content/mtime` `encoding/line endings` `patch` `temp path/mode`

Trigger	FileEditTool이 prior read 뒤 old/new string edit를 받음
Transition	expected string과 cardinality를 검사하고 sync reread한다. stale이면 equality proof가 없는 한 거부하고 patch를 계산해 flushed temp file을 rename한 뒤 read state를 갱신한다.
Invariant	• staleness check와 write 사이 await 없음 • 가능하면 mode와 symlink target behavior 보존 • ambiguous multi-match는 replace_all 요구
Failure / recovery	• stale content는 write 전 실패 • atomic replace 실패 시 flushed direct write • temp cleanup은 best effort

◆ why it matters

permission을 받았더라도 stale state면 effect를 멈춰야 한다는 점에서 authority와 concurrency control이 결합된다.



Exact project evidence

src/tools/FileEditTool/FileEditTool.ts:86 · src/tools/FileEditTool/FileEditTool.ts:387 · src/tools/FileEditTool/FileEditTool.ts:429 · src/tools/FileEditTool/FileEditTool.ts:465 · src/tools/FileEditTool/FileEditTool.ts:491 · src/utils/file.ts:362 · src/utils/file.ts:385 · src/utils/file.ts:434 · src/utils/file.ts:439

> 정적 source 관찰이며 runtime reachability와 current-product parity는 검증하지 않았다.

Reader question

Headless conversation owner와 SDK projection

UI가 없어도 누가 conversation과 terminal state를 소유하는가?

● observed_project_fact

QueryEngine은 request를 internal message로 바꾸고 durable checkpoint와 query() 실행을 순서화한 뒤 SDK event/result로 투영한다.

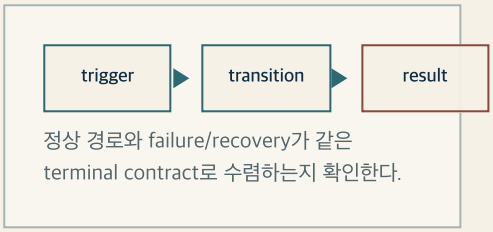
headless caller는 화면 state를 읽을 수 없다. 대화 owner가 mutable conversation과 abort, session, terminal status를 명시적으로 관리해야 한다.

`conversation messages` `app state` `tool context` `session identity` `abort controller` `SDK status`

Trigger	submitMessage 또는 ask가 headless SDK request를 받음
Transition	request를 internal message로 바꾸고 checkpoint를 persist한다. query()를 drive하고 conversation을 갱신한 뒤 internal message를 SDK shape로 normalize한다.
Invariant	• conversation mutation과 transcript checkpoint 순서 유지 • internal variant를 직접 leak하지 않고 projection • abort와 terminal reason이 generator를 닫는다
Failure / recovery	• orphan permission flow repair • compaction/tool result를 resume 전에 기록 • headless caller에는 UI가 아닌 terminal status

◆ why it matters

core turn semantics와 CLI/SDK surface를 분리해도 conversation ownership과 durability contract는 유지되어야 한다.



Exact project evidence
src/QueryEngine.ts:184 · src/QueryEngine.ts:209 · src/QueryEngine.ts:451 · src/QueryEngine.ts:675 · src/QueryEngine.ts:769 · src/QueryEngine.ts:1186 · src/utils/queryHelpers.ts:102 · src/utils/queryHelpers.ts:224

> 정적 source 관찰이며 runtime reachability와 current-product parity는 검증하지 않았다.

Reader question

Remote-command safety gate를 가진 입력 분류

같은 문자열이 prompt, command, attachment 중 무엇이 되는가?

● observed_project_fact

processUserInputBase는 media와 origin을 먼저 정규화하고 bridge-safe override를 평가한 뒤 bash, slash command 또는 ordinary prompt로 보낸다.

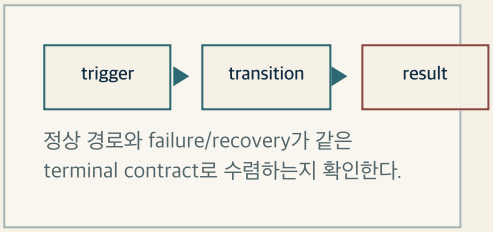
입력 text만 보면 intent를 알 수 없다. origin과 mode가 다르면 같은 slash text도 local effect를 요청하거나 plain prompt가 될 수 있다.

`input mode` `content blocks` `bridge origin` `slash skip flag` `attachments` `image metadata` `permission mode`

Trigger	raw string 또는 content block이 processUserInputBase에 들어옴
Transition	media를 normalize하고 bridge-safe slash override를 평가한다. feature keyword와 attachment를 처리한 뒤 bash, slash command 또는 ordinary prompt로 dispatch한다.
Invariant	• bridge client는 unsafe local command를 실행할 수 없다 • unknown remote slash text는 plain text로 fall through • image normalization이 prompt path보다 먼저
Failure / recovery	• unsafe remote command는 model query 없이 local-command message • mode/input mismatch는 early throw • hook output은 별도 cap

◆ why it matters

입력 경계에서 trust를 잃으면 이후 permission layer가 있어도 잘못된 control path에 진입할 수 있다.



Exact project evidence
src/utils/processUserInput/processUserInput.ts:281 · src/utils/processUserInput/processUserInput.ts:314 · src/utils/processUserInput/processUserInput.ts:422 · src/utils/processUserInput/processUserInput.ts:455 · src/utils/processUserInput/processUserInput.ts:516 · src/utils/processUserInput/processUserInput.ts:531 · src/utils/processUserInput/processUserInput.ts:576

> 정적 source 관찰이며 runtime reachability와 current-product parity는 검증하지 않았다.

Reader question

Bounded WebSocket reconnect 상태 기계

연결이 끊겼을 때 언제 다시 시도하고 언제 끝내는가?

● observed_project_fact

close code와 이전 state에 따라 retry budget을 나누고 explicit close에서는 timer를 정리해 reconnect가 영원히 계속되지 않게 한다.

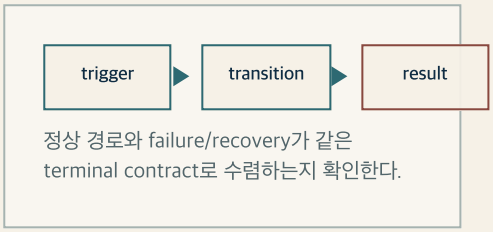
모든 disconnect를 같은 retry로 다루면 permanent rejection에도 반복하거나 이미 닫힌 session을 되살릴 수 있다.

`connecting/connected/closed` `general attempts` `4001 attempts` `ping/reconnect timers` `socket reference`

Trigger	session WebSocket close 또는 caller의 force reconnect
Transition	permanent code는 종료한다. 4001은 별도 bounded delay를 적용하고 다른 transient close는 connected state에서만 retry한다. explicit close는 timer를 취소하고 force reconnect는 budget을 reset한다.
Invariant	• permanent rejection은 retry하지 않는다 • explicit close에서 timer clear • send는 connected state에서만
Failure / recovery	• general retry는 5회 후 중단 • 4001은 3회 후 중단 • ping error는 close handler에 위임

◆ why it matters

recovery는 항상 재시도가 아니라 원인별 budget과 terminal decision을 갖는 상태 기계여야 한다.



Exact project evidence
src/remote/SessionsWebSocket.ts:18 · src/remote/SessionsWebSocket.ts:26 · src/remote/SessionsWebSocket.ts:34 · src/remote/SessionsWebSocket.ts:82 · src/remote/SessionsWebSocket.ts:232 · src/remote/SessionsWebSocket.ts:290 · src/remote/SessionsWebSocket.ts:369 · src/remote/SessionsWebSocket.ts:393

> 정적 source 관찰이며 runtime reachability와 current-product parity는 검증하지 않았다.

Cross-algorithm review

성공 경로보다 닫히는 규칙이 시스템을 설명한다

불변식	관련 알고리즘	깨졌을 때 보이는 현상	회복 수단
proposal/result pairing	turn, effect, normalization, resume	provider protocol 불일치, 다음 turn 실패	synthetic error result, orphan 제거, sibling 복구
bounded progress	turn, compaction, reconnect	무한 retry 또는 정지하지 않는 session	turn budget, retry cap, circuit breaker
ordered durability	query engine, transcript append	checkpoint가 message보다 앞서거나 parent chain 손실	per-file queue, ordered mutation
authority before effect	input dispatch, effect interpreter	remote origin 또는 denial이 실제 효과로 이동	safety gate, permission decision, structured denial
deterministic state fold	stream reducer, scheduler	event 순서나 동시성에 따라 context가 달라짐	index/type check, original-order modifier fold
stale-state rejection	file edit commit	읽은 뒤 바뀐 파일에 오래된 edit 적용	synchronous reread, equality proof, pre-write failure

● observed_project_fact

catalog가 직접 말하는 것

각 card는 trigger, state, transition, invariant, failure/recovery와 source locator를 가진다. source reference 89개는 존재·line-range 검사를 통과했다.

? unresolved

아직 검증하지 못한 것

runtime trace와 conventional test corpus는 없다. 따라서 실제 빈도, hot path, 성능, product parity는 이 불변식 표에서 유도할 수 없다.

Static structure

디렉터리는 많지만 의존 그래프는 큰 덩어리다

정적 import 분석은 “어디를 먼저 옮길까”에 답하지 않는다. 대신 변경 blast radius를 과소평가하면 안 된다는 경고를 준다.

1,884	13,219	65.25%
분석한 TypeScript/TSX 구현 파일	해결된 import occurrence	top-level group을 넘는 import occurrence
35 / 36	**1,433**	**591**
하나의 cyclic group SCC에 포함된 group	type-inclusive 최대 file SCC	runtime-like 휴리스틱 최대 file SCC



상위 cross-group edge는 components→utils 868회, tools→utils 696회, services→utils 669회다. 동시에 utils→services와 utils→tools도 관찰되어 단방향 layer로 단순화하기 어렵다.

해석 금지 SCC가 크다고 자동으로 결함인 것은 아니다. runtime-like graph도 실제 실행이 아니라 type-only edge를 제외한 정적 syntax 휴리스틱이다.

Metric interpretation

617개의 unresolved import는 617개의 버그가 아니다

● observed_project_fact

analyzer는 617 unresolved internal import occurrence와 250 raw specifier를 기록했다.

분석기는 TypeScript AST의 import/export와 literal dynamic import/require, relative와 `src/` specifier를 대상으로 한다. directory는 resolved module로 세지 않고, non-TypeScript file은 occurrence로만 처리한다.

module-topology.json · analyzer semantics · totals

? unresolved

unresolved의 원인은 현재 bundle만으로 분해할 수 없다.

누락된 feature-gated module, resolver 한계, generated file, specifier 이상이 함께 섞일 수 있다. 따라서 occurrence 수를 결함 수나 복구 가능 파일 수로 바꾸어 말하지 않는다.

◆ project_inference

왜 이 숫자가 중요한가

directory move를 첫 단계로 삼으면 type, UI, service, tool, root composition의 숨은 계약을 동시에 흔들 가능성이 크다. topology는 추출 순서를 결정하지 않지만, 먼저 behavior fixture와 compatibility facade가 필요하다는 운영 우선순위를 지지한다.

측정	말할 수 있음	말할 수 없음
cross-group occurrence	정적 참조 집중도	runtime call frequency
SCC	상호 도달 가능한 import graph	각 edge의 설계 결함 여부
unresolved specifier	현재 resolver가 찾지 못함	원본 제품에도 파일이 없음

Bounded candidates

파일 위치보다 “무엇을 소유하는가”로 다시 묶는다

여덟 후보는 현행 package나 upstream bounded context가 아니다. 실행 흐름과 import topology를 함께 읽기 위한 project inference다.

M1 · composition-routing

bootstrap, command/tool/surface 선택과 interactive/headless 조립을 맡는 가설. root fan-out이 커서 추출 위험은 높다.

`main.tsx` · `tools.ts` · `commands.ts` · `bootstrap/**` · `entrypoints/**`

M2 · turn-engine

user turn, model iteration, tool-result reinjection, limit와 terminal projection을 소유한다. cross-cutting context 때문에 위험이 높다.

`query.ts` · `QueryEngine.ts` · `queryHelpers.ts` · `context.ts`

M3 · effect-runtime

schema, hook, permission, concurrency, execution, result normalization으로 proposal을 effect로 해석한다.

`services/tools/**` · `hooks/**` · `Tool.ts`

M4 · built-in-effects-and-presenters

built-in tool과 UI/terminal presentation 계약을 구현한다. surface가 넓지만 runtime 경계와 분리해서 보아야 한다.

`tools/**` · `components/**` · `ink/**`

> **계속** 다음 면은 durability, extension, surface, platform 책임을 분리하고 후보 간 의존을 읽는다.

SEMANTIC-MODULES · CONTINUED · M5–M8

기록, 확장, 표면, 플랫폼은 서로 다른 변화 이유를 가진다

M5 · session-recovery

append-only transcript, parent-linked reconstruction, compaction, resume topology repair를 묶는다. round-trip 의미가 extraction 위험이다.

`sessionStorage.ts · services/compact/** · SessionMemory/**`

M6 · extension-runtime

plugin, skill, MCP capability, hook extension을 발견하고 호스팅한다. feature gate와 provider variance가 크다.

`plugins/** · skills/** · mcpClient* · hooks/**`

M7 · surface-adapters

CLI, bridge, remote, SDK, server transport를 core turn/effect semantics에 맞춘다. transport별 차이를 adapter 책임으로 본다.

`cli/** · bridge/** · remote/** · server/**`

M8 · platform-adapters

filesystem, process, config, telemetry, permission, OS, provider mechanism을 제공한다. `utils` 전체를 하나의 모듈로 취급해서는 안 된다.

`utils/** · services/api/** · analytics/** · native-ts/**`



M1은 이들을 조립하고, M6은 확장을 연결하며, M8은 여러 후보에 platform mechanism을 제공한다. 화살표는 분석용 책임 관계이며 현행 package boundary가 아니다.

반증 조건 runtime trace가 책임 mapping과 충돌하거나 공식 ownership evidence가 도착하면 후보 집합은 invalidation 대상이다.

How to read

진입점보다 먼저 불변식을 고르고 양쪽을 따라간다

대형 SCC에서는 "main부터 순서대로" 읽기가 쉽게 분산된다. 하나의 contract를 고르고 producer, transformer, consumer, durable record를 왕복하는 편이 낫다.

질문	시작 지점	따라갈 경계	도착 확인
turn은 왜 계속되는가?	`query.ts`	stream → tool result → terminal reason	bounded continuation 또는 terminal
요청이 언제 effect가 되는가?	`toolExecution.ts`	schema → hook → permission → executor	structured result와 post-hook
partial stream은 언제 message가 되는가?	`claude.ts`	block index/type → delta → late usage	queued transcript reference
중단 뒤 무엇을 복구하는가?	`sessionStorage.ts`	UUID map → parent chain → compact/snip repair	bounded leaf-to-root chain
동시 tool은 state를 어떻게 합치는가?	`toolOrchestration.ts`	safety partition → bounded batch → ordered fold	deterministic context

읽기 규칙
source locator를 확인한 뒤 failure branch를 먼저 찾고, 해당 실패를 다음 layer가 어떤 result로 관찰하는지 확인한다.

피해야 할 규칙
파일 수가 적거나 directory 이름이 좋아 보인다는 이유로 extraction 후보를 정하지 않는다.

Work-item contract

첫 산출물은 이동된 디렉터리가 아니라 재현 가능한 behavior fixture다

관례적 test/spec corpus가 관찰되지 않은 snapshot에서는 구조 개선의 안전성을 기존 테스트 수로 가정할 수 없다. 먼저 무엇을 보존할지 실행 가능한 계약으로 만든다.

◆ project_inference

한 work item이 가져야 할 것

1. 하나의 observable contract 또는 invariant
2. 정확한 source revision과 evidence line
3. success와 failure fixture
4. owner와 consumer, non-goal
5. targeted → differential → wider check
6. rollback 또는 compatibility 기간

● observed_project_fact

현재 증거가 요구하는 제약

bounded filename scan에서 conventional test/spec file은 관찰되지 않았다. 이는 snapshot 밖에 테스트가 없었다는 증명이 아니다. build와 runtime도 실행하지 않았다.

따라서 repository-wide failure를 곧바로 candidate 결함으로 귀속하지 않고 candidate-related, pre-existing, environment/tool, unresolved로 분류해야 한다.



검증 계층은 작은 shape와 static check에서 시작해 behavior contract와 변경 전후 trace로 확장한다. 넓은 system check는 마지막에 수행한다.

Claim lifecycle

worker_done과 clean worktree는 결론의 증거가 아니다

◆ project_inference

claim과 lifecycle을 직교시킨다

claim class는 source-observed, repository-authored, bounded-inference, unknown으로, lifecycle은 pending, observed, corroborated, candidate, rejected, superseded로 관리한다. “무슨 종류의 주장인가”와 “검토가 어디까지 왔는가”는 다른 축이다.

● observed_project_fact

운영 상태가 의미하지 않는 것

runtime session start는 Agent activation이 아니고, worker contribution은 acceptance가 아니며, report 작성은 publication이 아니다. Git/worktree clean과 worker_done도 settlement 증거가 아니다.

상태	뜻	자동으로 뜻하지 않는 것
pass	지정 check가 해당 revision에서 통과	전체 정확성·acceptance
hold	필요 증거 없음 또는 source 변경	실패·포기
fail	재현 가능한 contract 위반	인접 artifact의 거절
candidate	검토 가능한 제안	공식 경계·publication

Revision rule 코드와 문서가 같은 revision/digest를 가리키지 않으면 어느 한쪽도 자동으로 최신이라고 간주하지 않는다. material change는 predecessor를 덮어쓰지 않고 successor record로 남긴다.

Evolution strategy

한 번에 하나의 concrete dependency를 뒤집는다

큰 SCC를 한 번에 해체하려 하지 않고, 입력 · 출력 · 실패 상태를 완전히 기술할 수 있는 leaf policy에서 시작한다.

01 FIXTURE	turn, stream, tool pairing, transcript/resume의 정상·실패 결과를 characterization corpus로 고정한다.
02 PURE SEAM	message normalization, pairing repair, scheduling policy, compaction decision처럼 I/O 없이 설명 가능한 policy를 분리한다.
03 PORT	filesystem, provider, process, UI를 port 뒤 adapter로 밀어낸다. compatibility facade로 기존 caller를 보존한다.
04 INVERT	한 번에 하나의 concrete dependency를 역전하고 event, terminal reason, transcript를 differential 비교한다.
05 SLICE	하나의 surface 또는 extension만 vertical slice로 이동한다. package boundary와 rollback을 먼저 정한다.
06 RETIRE	consumer가 새 경계를 사용하고 wider fitness가 안정된 뒤에만 barrel, shim, old surface를 제거한다.

> **높은 위험의 첫 수** `query.ts`, 전체 `utils`, root composition을 처음부터 package로 분리하지 않는다. fan-in이 높은 이름은 경계가 아니라 조사 대상이다.

Report-only phases

A와 B: behavior를 고정하고 실제 경로를 측정한다

◆ project_inference

Phase A · Characterization baseline

turn terminal-reason table, stream event projection, tool_use/tool_result pairing, transcript append/resume/compaction round-trip corpus를 만든다. 정상뿐 아니라 abort, partial stream, orphan tool use, context overflow를 포함하고 source revision과 fixture provenance를 묶는다.

◆ project_inference

Phase B · Runtime trace map

CLI 시작부터 첫 terminal result, permission/hook/execution, transcript write와 resume read를 bounded trace로 측정한다. prompt body와 개인정보는 저장하지 않는다. 실행할 수 없는 feature는 unknown으로 둔다.



static topology는 trace의 질문을 만들고, runtime trace는 실제 hot path와 비활성 edge를 구분한다. 둘의 차이가 contract seam 후보를 좁힌다.

실행 상태 이 phase들은 report-only 후보이며 owner, plan, transition record가 배정되지 않았다.

ROADMAP · CONTINUED · PHASES C–E

C, D, E: 계약을 추출하고 한 slice로 검증한다

Phase C · Contract seams

API normalization과 pairing repair → tool concurrency partition → compaction decision/retry budget → file-edit stale/atomic policy → reconnect/backoff 순으로 검토한다. 각 후보에 caller map, adapter dependency, differential fixture, compatibility facade가 필요하다.

Phase D · One vertical slice pilot

단일 surface 또는 extension, 제한된 consumer, 되돌릴 수 있는 facade로 좁힌다. package-scoped 검증 뒤 wider fitness를 실행하고 end-to-end 의미가 같을 때만 다음 slice를 연다.

Phase E · Governance and maintenance

source revision이 바뀔 때 topology analyzer와 evidence line drift 검사를 다시 실행한다. dependency budget, fixture coverage, unresolved import trend, claim supersession을 revision-bound record로 관리한다.

단계	진입 증거	중단 조건
C	A/B fixture와 trace	behavior를 완전히 기술할 수 없음
D	seam, facade, rollback	differential mismatch
E	반복 가능한 analyzer/check	source identity 또는 owner 불명

> **Page policy와 같은 원칙** 단계 수도 목표가 아니다. 필요한 evidence가 없으면 다음 phase로 압축해서 넘어가지 않고 hold한다.

Explicit unresolved

현재 snapshot으로 답할 수 없는 질문

? unresolved

질문	왜 모르는가	필요한 후속 증거
공식 module owner는 누구인가?	공식 ownership map이 snapshot에 없음	upstream owner document 또는 accepted boundary record
실제 production hot path는 무엇인가?	runtime trace를 수집하지 않음	privacy-safe bounded trace
누락 모듈이 어떤 behavior를 바꾸는가?	feature-gated source가 불완전함	추가 source 또는 controlled build comparison
12개 알고리즘이 현재 제품에도 동일한가?	제3자 복구 snapshot만 분석	현재 공식 source/runtime evidence
어떤 seam이 가장 먼저 안전하게 추출되는가?	fixture와 differential trace가 아직 없음	Phase A-C 결과
실제 장애율과 성능 병목은 어디인가?	metrics와 incident evidence가 없음	operational telemetry와 incident review

◆ project_inference

unknown은 보고서의 결함이 아니라 다음 조사에서 지켜야 할 boundary다.

후속 조사의 목적은 모든 빈칸을 채우는 것이 아니라, 독자의 결정에 필요한 빈칸을 정확한 revision과 source class로 좁히는 것이다.

알고리즘 01 – 06 근거 색인

각 locator는 frozen revision의 project-analysis catalog에서 가져왔다. 링크는 commit에 고정되어 있으며 network reachability는 별도 검증 대상이다.

● ALG-TURN-LOOP

Generator 기반 turn 상태 기계

src/query.ts:219 · src/query.ts:241 · src/query.ts:554 · src/query.ts:834 · src/query.ts:1012 · src/query.ts:1062 · src/query.ts:1381 · src/query.ts:1506 · src/query.ts:1705

● ALG-EFFECT-INTERPRETER

Tool proposal을 허가된 effect로 해석하기

src/services/tools/toolExecution.ts:337 · src/services/tools/toolExecution.ts:599 · src/services/tools/toolExecution.ts:916 · src/services/tools/toolExecution.ts:995 · src/services/tools/toolExecution.ts:1571 · src/services/tools/toolHooks.ts:39 · src/services/tools/toolHooks.ts:332 · src/services/tools/toolHooks.ts:435

● ALG-STREAM-REDUCER

Provider stream reducer와 late finalization

src/services/api/claude.ts:752 · src/services/api/claude.ts:1017 · src/services/api/claude.ts:1995 · src/services/api/claude.ts:2053 · src/services/api/claude.ts:2213 · src/services/api/claude.ts:2229 · src/services/api/claude.ts:2309

● ALG-TRANSCRIPT-APPEND

Queued append-only JSONL journal

src/utils/sessionStorage.ts:532 · src/utils/sessionStorage.ts:606 · src/utils/sessionStorage.ts:634 · src/utils/sessionStorage.ts:841 · src/utils/sessionStorage.ts:993 · src/utils/sessionStorage.ts:1128 · src/utils/sessionStorage.ts:1408

● ALG-RESUME-GRAPH

Transcript graph 재구성과 repair

src/utils/sessionStorage.ts:1839 · src/utils/sessionStorage.ts:1982 · src/utils/sessionStorage.ts:2069 · src/utils/sessionStorage.ts:2097 · src/utils/sessionStorage.ts:2224 · src/utils/sessionStorage.ts:3472 · src/utils/sessionStorage.ts:3704

● ALG-API-NORMALIZATION

Provider message compiler와 tool-pair repair

src/services/api/claude.ts:1266 · src/services/api/claude.ts:1298 · src/utils/messages.ts:1989 · src/utils/messages.ts:5123 · src/utils/messages.ts:5305 · src/services/api/claude.ts:3063

ALGORITHM-INDEX · CONTINUED · 07–12

알고리즘 07 – 12 근거 색인

각 locator는 frozen revision의 project-analysis catalog에서 가져왔다. 링크는 commit에 고정되어 있으며 network reachability는 별도 검증 대상이다.

● ALG-TOOL-SCHEDULER

입력 의존 tool scheduling과 deterministic fold

src/services/tools/toolOrchestration.ts:8 · src/services/tools/toolOrchestration.ts:19 · src/services/tools/toolOrchestration.ts:91 · src/services/tools/toolOrchestration.ts:118 · src/services/tools/toolOrchestration.ts:152 · src/services/tools/StreamingToolExecutor.ts

● ALG-COMPACTION-MIGRATION

Bounded retry를 가진 lossy context migration

src/services/compact/autoCompact.ts:70 · src/services/compact/autoCompact.ts:160 · src/services/compact/autoCompact.ts:241 · src/services/compact/compact.ts:227 · src/services/compact/compact.ts:243 · src/services/compact/compact.ts:387 · src/services/compact/compact.ts:713

● ALG-FILE-EDIT-COMMIT

Optimistic file edit와 atomic-replace 시도

src/tools/FileEditTool/FileEditTool.ts:86 · src/tools/FileEditTool/FileEditTool.ts:387 · src/tools/FileEditTool/FileEditTool.ts:429 · src/tools/FileEditTool/FileEditTool.ts:465 · src/tools/FileEditTool/FileEditTool.ts:491 · src/utils/file.ts:362 · src/utils/file.ts:385 · src/utils/file.ts:434 · src/utils/file.ts:439

● ALG-QUERY-ENGINE-PROJECTION

Headless conversation owner와 SDK projection

src/QueryEngine.ts:184 · src/QueryEngine.ts:209 · src/QueryEngine.ts:451 · src/QueryEngine.ts:675 · src/QueryEngine.ts:769 · src/QueryEngine.ts:1186 · src/utils/queryHelpers.ts:102 · src/utils/queryHelpers.ts:224

● ALG-INPUT-DISPATCH

Remote-command safety gate를 가진 입력 분류

src/utils/processUserInput/processUserInput.ts:281 · src/utils/processUserInput/processUserInput.ts:314 · src/utils/processUserInput/processUserInput.ts:422 · src/utils/processUserInput/processUserInput.ts:455 · src/utils/processUserInput/processUserInput.ts:516 · src/utils/processUserInput/processUserInput.ts:531 · src/utils/processUserInput/processUserInput.ts:576

● ALG-REMOTE-RECONNECT

Bounded WebSocket reconnect 상태 기계

src/remote/SessionsWebSocket.ts:18 · src/remote/SessionsWebSocket.ts:26 · src/remote/SessionsWebSocket.ts:34 · src/remote/SessionsWebSocket.ts:82 · src/remote/SessionsWebSocket.ts:232 · src/remote/SessionsWebSocket.ts:290 · src/remote/SessionsWebSocket.ts:369 · src/remote/SessionsWebSocket.ts:393

Reader reference

이 책에서 반복되는 경계 언어

turn
한 model request만이 아니라 stream, tool execution, result reinjection과 terminal reason까지 포함하는 실행 단위.

effect
filesystem·process·network처럼 외부 관찰 가능한 상태를 바꾸는 일. tool proposal과 구분한다.

closure
tool_use가 성공·실패·거부·abort 중 하나의 tool_result로 닫히는 protocol 조건.

projection
내부 state나 message를 UI, SDK, provider format으로 변환한 표현.

journal
순서와 parent link를 보존해 다시 읽을 수 있는 append 기록.

compaction
token 압력에서 일부 context를 summary와 preserved attachment로 옮기는 손실 변환.

SCC
directed graph에서 서로 도달 가능한 node 집합. 자동으로 설계 결함을 뜻하지 않는다.

compatibility facade
새 경계로 이동하는 동안 기존 caller contract를 유지하는 임시 표면.

정보 배치 규칙

introduction	보고서 전체를 읽는 모델을 바꿀 때	worked trace	순서와 중간 상태가 핵심일 때
prerequisite callout	인접 주장을 위한 짧은 배경일 때	failure case	경계가 실패로 더 선명해질 때
glossary	안정된 용어의 lookup이 필요할 때	appendix	main path에 필수적이지 않은 검증 깊이

> mandatory prerequisite를 page 수 때문에 appendix로 보내지 않는다. page count는 결과이며, 의미를 줄이는 cap이 아니다.

What the reader should carry forward

이 프로젝트는 tool 목록이 아니라 state · authority · durability의 왕복으로 읽힌다

turn engine은 계속할지 끝낼지를 결정하고, stream reducer는 partial observation을 message로 수렴시키며, effect runtime은 요청이 외부 변화가 될 자격을 판단한다.

실행을 이해하는 질문

trigger가 무엇인지, state를 누가 보유하는지, 어떤 transition이 authority를 넘기는지, failure가 result인지 abort인지, recovery가 retry인지 repair인지 묻는다.

특히 tool_use/tool_result closure, bounded retry, ordered transcript, stale-state rejection은 서로 다른 파일에 있어도 같은 품질 원칙을 공유한다.

구조를 바꾸는 질문

directory보다 contract seam을 먼저 찾고, behavior fixture와 differential trace를 만든다. 의미 모듈 후보는 소유권 선언이 아니라 이 질문을 분리하기 위한 지도다.

다음 단계는 큰 이동이 아니라 하나의 pure policy, 하나의 adapter, 하나의 vertical slice를 증거와 함께 바꾸는 일이다.

좋은 분석은 모든 것을 확정하지 않는다. 무엇을 관찰했고, 무엇을 추론했으며, 무엇이 아직 필요한지를 다음 사람에게 정확히 넘긴다.

Candidate delivery 이 PDF는 successor authoring contribution이다. 독립 factual/visual/reader validation, acceptance, publication, integration, settlement는 수행되지 않았다.

Delivery basis

이 보고서를 다시 확인하는 방법

● observed_project_fact

고정 authoring input

authoring freeze SHA-256은 `1fe79995a7160c2bb20fc8b46e16d4e0669daf57ca48be32a16ad7689baf2d4e`다. project-analysis manifest와 모든 required/limited input digest, 그리고 manifest가 묶은 14개 source-analysis artifact digest를 작성 전에 대조했다.

주요 분석 산출물

- `PROJECT-ANALYSIS.md` — source-bound synthesis
- `algorithm-catalog.json` — 12 algorithm cards와 89 evidence locator
- `module-topology.json` — 정적 import graph와 limitations
- `semantic-module-candidates.json` — 책임 기반 후보, 공식 경계 아님
- `project-operating-model.md` — fixture와 evidence-led change model
- `continuation-roadmap.md` — report-only Phase A-E

▤ external_background

MDN Generator · MDN yield · Anthropic tool use. 일반 개념만 지원하며 project behavior를 증명하지 않는다.

? unresolved

복구 snapshot의 누락, build/runtime parity, feature flag, 공식 ownership, production behavior, physical print, grayscale, accessibility, reader comprehension은 독립 검증 전까지 보류한다.

> 이 문서는 activation·formal acceptance·publication·promotion·integration·settlement 권한을 전달하지 않는다.